PROJECT AND TEAM INFORMATION

Project Title

5G SDN: DYNAMIC NETWORK MANAGEMENT

Student / Team Information

Team Name:	*Tech_Pack*
Team member 1 (Team Lead) *Name:- Sarthak Ghildiyal* *Student ID:- 2510520136* *Email:- sarthakghildiyal07@gmail.com*	
Team member 2 *Name:- Shivam Pharswan* *Student ID:- 2510520116* *Email:- 2510520116@geu.ac.in*	
Team member 3 *Name:- Aarav Revasiya* *Student ID:- 2510520183* *Email:- aaravrevasiya@gmail.com*	

PROPOSAL DESCRIPTION

Motivation

Modern 5G networks handle network traffic by differentiating it into many different categories. This includes high data like videos and redundant background requests. All packets are not treated equally as they might introduce unnecessary delays in transmission.
This project aims to create a 5g network simulation and show how network packets are prioritized , stored and classified based on their type.
These types include:
uRLLC : Ultra Reliable Low latency Communication-> Emergency control systems , industrial machines
eMBB: Enhanced Mobile Broadband-> Video Streaming , Large downloads
mIoT: Sensors and other devices
Local: Ordinary traffic , mostly redundant
The project will use fixed buffer to store packets and scheduler to decide what packet would be processed next. A User interface will display the packets , their priorities their status and order through percentages and colour coding.

State of the Art / Current solution

Real 5G networks utilize complex and specialized hardware to perform these tasks such as network slicing, Quality of service and scheduling to manage it. Network slicing allows same network to support different services while QoS manage system priorities. Load balancing then distributes these processes accordingly to avoid components from becoming overloaded.
Our project aims to demonstrate how these complex systems work through a network simulation in python through most common data structures. It allows packet , classification , scheduling and load balancing through an easy to understand GUI.

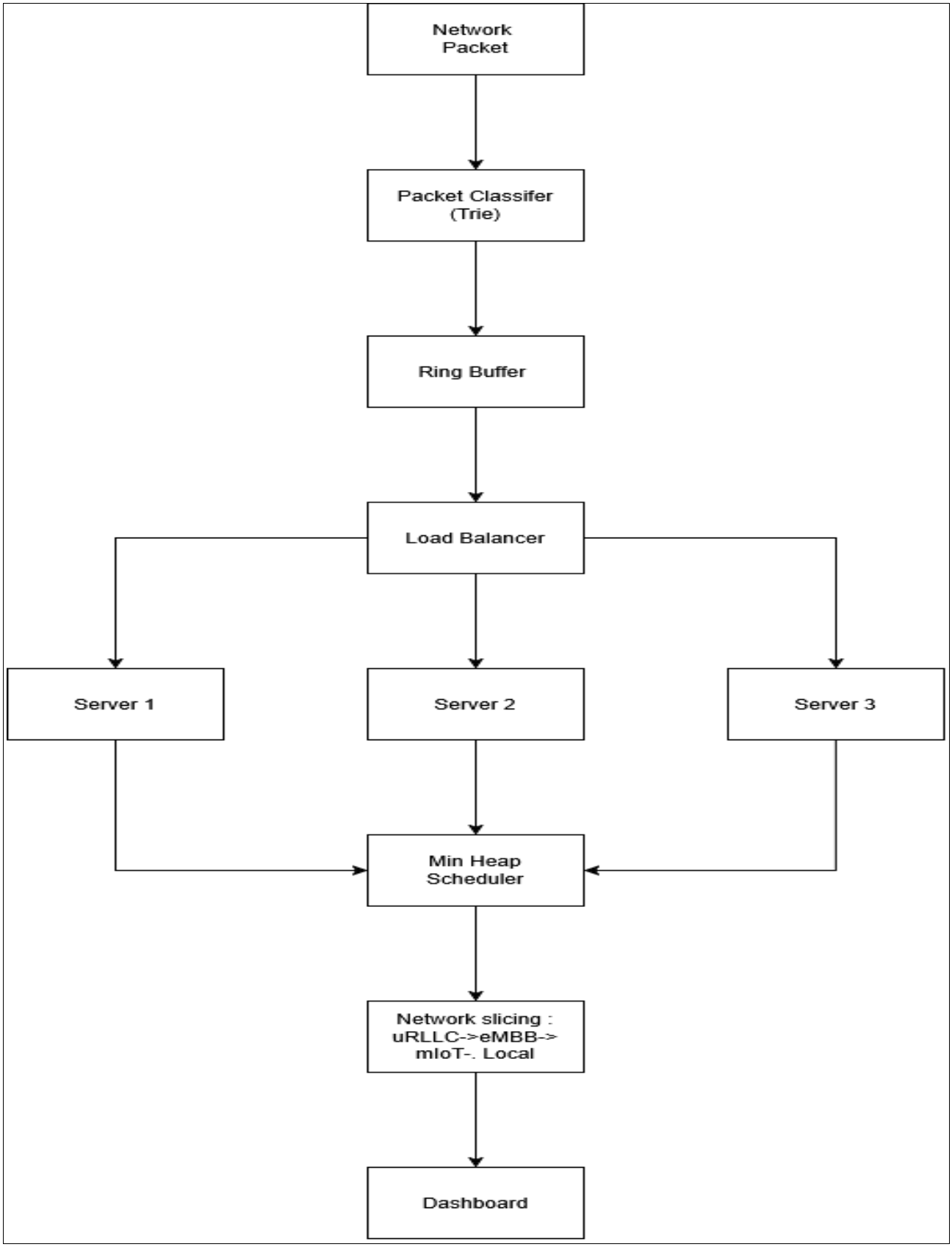
Project Goals and Milestones

1. Implement Network Simulation.
2. Implement a Trie for packet Classification.
3. Implement a Circular Ring buffer.
4. Implement Min heap for priority scheduling.
5.Implement Load Balancer to distribute packets between multiple servers.
6. Implement GUI to demonstrate the process using Streamit.
7. Implement graphics and visualizations.

Project Approach

The project will be developed in Python.
A traffic generator will create simulated IPv4 packets.
Packets will contain IP Address, ID , type.
A Trie (Tree based) will be used to classify packets based on their IP.
After classification they will be assigned their priority based on their type.
Packets will then enter a Circular Ring buffer.
A load Balancer then monitor the packet and allocates it to the suitable server.
A Min heap is used to schedule and determine which data packet will be processed first.
Order: uRLLC -> eMBB -> mIoT -> Local
StreamLit will be used to design a dashboard and display all the data through colour coding and visualization.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

The project will deliver a working Python Based SDN simulator.
Main Deliverables:
1. Packet Generator
2. A Trie Based Packet Classifier
3. A circular ring buffer for temp packet storage.
4. A priority scheduler through min heap
5. A Streamlit Dashboard displaying all the data
6. Colour Coded packet priority
7. Packet and scheduling information through visualization.

Assumptions

The Project assumes packets are all simulated inside the program and no real 5G physical networking equipment is used. The system will use iPv4 addressed only. Traffic catagories and prorities shall remain fixed during the process.

References

1. *3GPP TS 23.501, System Architecture for the 5G System (Release 16/17)*
2. *Cormen, T. H., et al. Introduction to Algorithms, 4th Edition. MIT Press.*
3. *Varghese, G. (2005). Network Algorithmics: An Interdisciplinary Approach to Designing Fast Networked Devices. Morgan Kaufmann.*
4. *GeeksForGeeks Trie Data sctructures , Circular queue , Heap data structure , load balancing.*